K.R. MANGALAM UNIVERSITY
School of Engineering and Technology (SOET)

Make the Computer Draw a Box

Nand2Tetris Capstone Project Report

Hack computer (HDL) · Python Hack assembler · Screen program for K = 02

Student	**Manas Aggarwal**	**Roll number**	**2501730302**
Programme	B.Tech Computer Science and Engineering – Artificial Intelligence and Machine Learning	**University / School**	K.R. Mangalam University, School of Engineering and Technology (SOET)
Personal number K	02 (last two digits of the roll number)	**Box (derived from K)**	Starts at row 2 · 20 rows tall · 48 px (3 words) wide
Course	Nand to Tetris – I (ETCCNT301)	**Extra assembler feature**	K mod 3 = 2 → warning for a label declared twice

Abstract

This report documents the design, implementation and validation of a complete Hack computing stack for the Nand2Tetris capstone. In Part A, the CPU, Memory and Computer chips were written in HDL and checked with the five official test scripts. In Part B, a two-pass Hack assembler was written in Python; its output was compared with the official assembler for Add, Max, Rect and Pong, and five built-in self-tests were run, including the duplicate-label warning required for K = 02. In Part C, Box.asm, assembled with the student's own assembler, draws a 48 × 20-pixel black box starting at screen row 2 while a key is held and clears it when the key is released, as observed in the CPU Emulator.

Project objectives

1. Integrate CPU, Memory and Computer HDL chips into a working Hack computer that passes CPU.tst, Memory.tst, ComputerAdd.tst, ComputerMax.tst and ComputerRect.tst.
2. Implement a two-pass Hack assembler in Python whose output matches the official assembler on four reference programs, with the extra feature for K = 02.
3. Write Box.asm for K = 02 and translate it to Box.hack with the student's own assembler, not the official one.
4. Run the program in the CPU Emulator and demonstrate the key-controlled draw and clear behaviour with recorded evidence.

Overall project architecture

The three parts form one toolchain (Fig. 1). The assembler turns the box program into machine code, and the Hack computer executes it by polling the keyboard word (address 24576) and writing the screen map (from address 16384). In Part C, Box.hack was run in the CPU Emulator; Computer.hdl was validated separately with the official Part A tests.

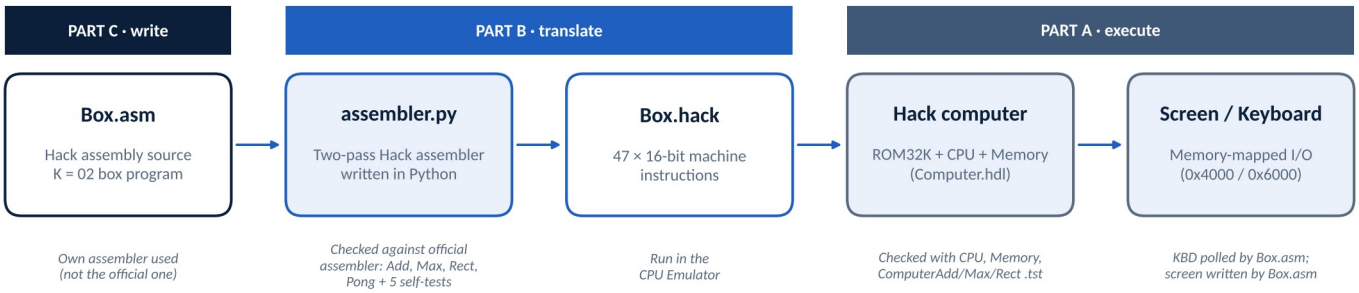


Fig. 1 Overall project architecture: the box program (Part C) is written in Hack assembly, translated by the Python assembler (Part B) and executed on the Hack computer built in HDL (Part A). Schematic diagram.

Implementation and evidence summary

Part	Implemented (supplied source)	Evidence supplied	Outcome recorded
A	CPU.hdl, Memory.hdl, Computer.hdl	5 Hardware Simulator screenshots (Figs. 3–7)	All five official tests end with "Simulation successful".
B	assembler.py (two passes, K = 02 duplicate-label warning); Add, Max, Rect, Pong .hack	4 official-assembler comparisons and 1 terminal capture (Figs. 9–13)	4 of 4 "Comparison successful"; 5 of 5 self-tests PASSED.
C	Box.asm → Box.hack (47 instructions), assembled with assembler.py	3 CPU Emulator screenshots (Figs. 15–17)	Box drawn while a key is held; cleared after release.

PART A · Hardware Integration

CPU.hdl · Memory.hdl · Computer.hdl — validated with the five official test scripts

The Hack computer is built from three chips (Fig. 2). **Computer.hdl** wires ROM32K, CPU and Memory together; the CPU sees only a 16-bit instruction and one data word, so all other behaviour comes from how it drives Memory.

CPU.hdl. Bit 15 separates A- from C-instructions; A-instructions load the A register through a Mux16. For C-instructions, bit 12 selects A or inM as the ALU's y input, bits 11–6 drive the ALU control pins, bits 5–3 become loadA, loadD and writeM, and bits 2–0 combine with the ALU flags zr and ng into the PC load signal. The outputs outM, writeM and addressM go to Memory; pc addresses ROM.

Memory.hdl. A DMux4Way on address[13..14] steers load to RAM16K (0x0000–0x3FFF) or Screen (0x4000–0x5FFF), a Mux4Way16 returns the selected word, and Keyboard is read at 0x6000.

Computer.hdl. ROM32K(address = pc) supplies the instruction; CPU(outM, writeM, addressM) drives Memory(in, load, address), whose out returns to the CPU as inM. Setting reset = 1 restarts execution at ROM[0].

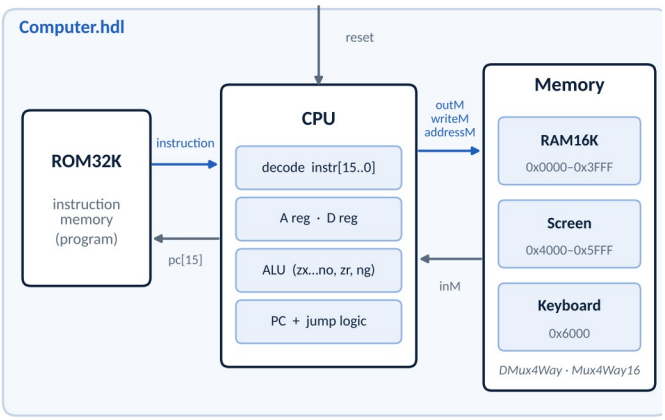


Fig. 2 Block diagram of Computer.hdl as wired in the supplied HDL. Schematic diagram.

Official test cases

Fig.	Test script	What it validates	Steps / outputs	Result shown
3	**CPU.tst**	Instruction decoding, A and D registers, ALU functions, writeM / addressM, jumps and reset, compared cycle by cycle with the compare file.	50 / 93	Simulation successful
4	**Memory.tst**	Address decoding and read/write across RAM16K, the screen map (0x4000–0x5FFF) and the keyboard word (0x6000).	1200 / 68	Simulation successful
5	**ComputerAdd.tst**	Whole computer running Add.hack (R0 = 2 + 3); a second run checks that reset restarts the PC.	16 / 15	Simulation successful
6	**ComputerMax.tst**	Whole computer running Max.hack (R2 = max(R0, R1)) with inputs that exercise different branches.	29 / 28	Simulation successful
7	**ComputerRect.tst**	Whole computer running Rect.hack: with R0 = 4 it draws 4 rows of 16 pixels in screen memory.	68 / 65	Simulation successful

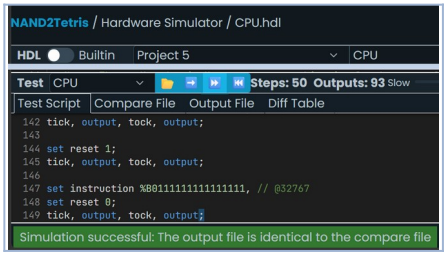


Fig. 3 CPU.hdl under CPU.tst (Hardware Simulator, Builtin off). Cropped: title bar and result panel.

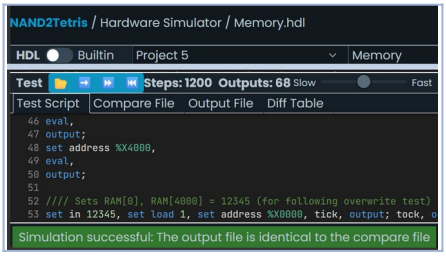


Fig. 4 Memory.hdl under Memory.tst. Cropped: title bar and result panel.

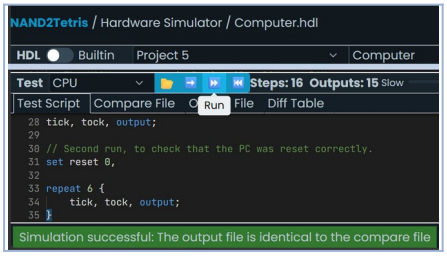


Fig. 5 Computer.hdl running Add.hack (ComputerAdd.tst). Cropped: title bar and result panel.

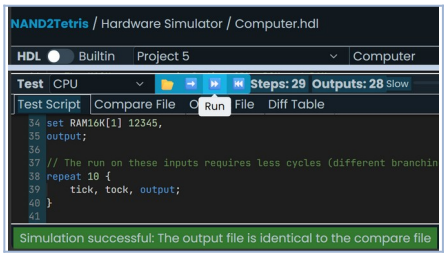


Fig. 6 Computer.hdl running Max.hack (ComputerMax.tst). Cropped: title bar and result panel.

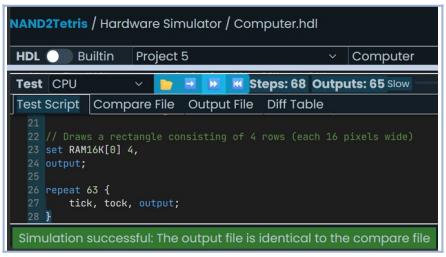


Fig. 7 Computer.hdl running Rect.hack (ComputerRect.tst). Cropped: title bar and result panel.

Reading the evidence

- The Builtin toggle is off in all five captures, so the student's HDL was the chip under test.
- Each capture ends with the simulator's green banner: output identical to the compare file.
- Chip sources: Screen, Keyboard and ROM32K are course-supplied built-ins; ALU, ARegister, DRegister, PC and RAM16K are instantiated from the project chip library.
- Design note: Memory.hdl does not decode addresses above 0x6000 separately (the specification treats them as invalid); they alias the keyboard word.

PART B · Python Hack Assembler

assembler.py · two-pass translation · K = 02 extra feature: duplicate-label warning

assembler.py translates a .asm file to a .hack file in two passes (Fig. 8).

Source cleaning. clean_source() removes // comments, trims whitespace and drops blank lines, keeping the original line number of every surviving line for error and warning messages.

Pass 1: symbols. first_pass() starts with the 23 predefined symbols (SP, LCL, ARG, THIS, THAT, SCREEN = 16384, KBD = 24576 and R0–R15). A (LABEL) line records the current ROM address and emits no code; every other line advances the ROM counter. If a label is already in the table, the assembler prints "Line n: Duplicate label …", keeps the first address and carries on. This is the K = 02 feature (Listing 1).

Pass 2: code. second_pass() produces 16 bits per instruction. @n or @symbol becomes 0 followed by a 15-bit address; a symbol missing from the table is a variable, allocated from RAM address 16 upward in order of first use. A C-instruction becomes 111 + comp (a-bit and c1–c6) + dest (3 bits) + jump (3 bits), looked up in the COMP, DEST and JUMP tables. The .hack file is written only if no errors remain.

Implemented but not covered by the recorded tests: malformed lines produce "Line n: …" error messages instead of a crash.

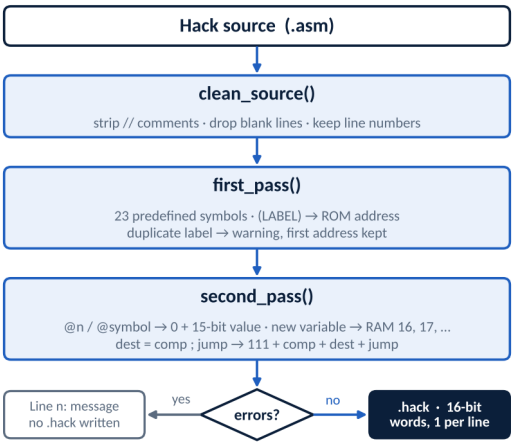


Fig. 8 Assembler workflow as implemented in assembler.py. Schematic diagram.

Internal self-tests

#	Self-test	What is checked	Result
1	Addition translation	@2, D=A, @3, D=D+A, @0, M=D → six expected words	PASSED
2	Variable allocation	@counter → 16 and @result → 17	PASSED
3	Duplicate-label warning	(LOOP) declared twice → 1 warning, LOOP stays at ROM 1	PASSED
4	Label address resolution	(START) after two instructions resolves to 2	PASSED
5	C-instruction translation	D=A, D=D+A, M=D, 0;JMP → expected binary	PASSED

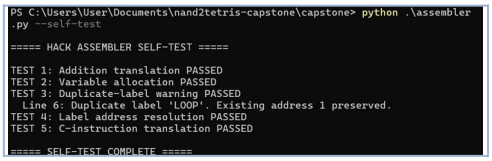


Fig. 9 Terminal capture of assembler.py --self-test: 5 tests PASSED; K = 02 warning shown.

Listing 1 assembler.py, first_pass(): duplicate-label handling

```
if label in symbols:
    warnings.append(
        f"Line {line_number}: Duplicate label "
        f"'{label}'. Existing address "
        f"{symbols[label]} preserved."
    )
    continue
symbols[label] = rom_address
```

Official comparison with the Nand2Tetris assembler (all four: "Comparison successful")

The web IDE's official assembler output was compared with the .hack file produced by assembler.py; re-running assembler.py on the supplied sources regenerates all four files with identical content.

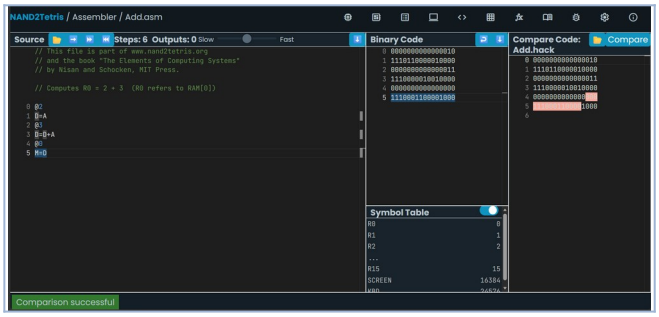


Fig. 10 Add.asm · 6 instructions · no labels or variables

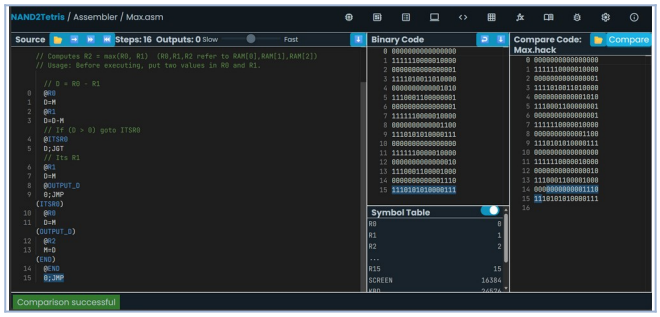


Fig. 11 Max.asm · 16 instructions · labels ITSR0, OUTPUT_D, END

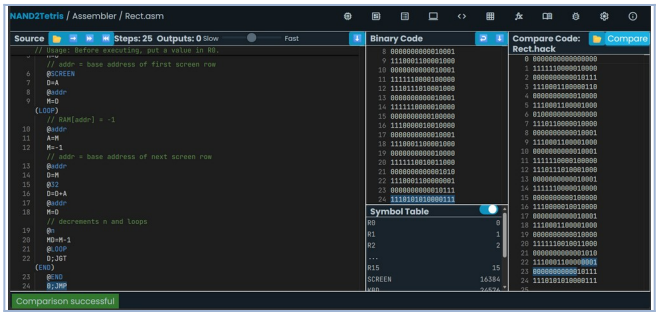


Fig. 12 Rect.asm · 25 instructions · n, addr → RAM 16, 17

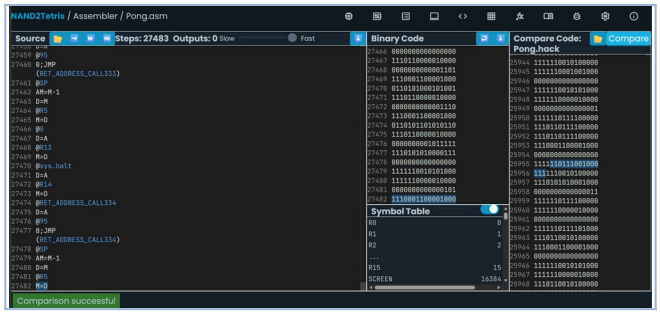


Fig. 13 Pong.asm · 27,483 instructions · symbols like sys.halt

PART C · Make the Computer Draw a Box

Box.asm · Box.hack · K = 02: row 2, 20 rows tall, 48 pixels wide · tested in the CPU Emulator

K calculation and rectangle parameters

K	Last two digits of 2501730302, so K = 02
Start row	K = 2: first word at SCREEN + 32 × 2 = 16384 + 64 = 16448
Height	20 rows (screen rows 2 to 21)
Width	16 × ((K mod 4) + 1) = 16 × 3 = 48 px = 3 words per row
Row stride	32 words per row: 3 written, then 29 skipped
Colour	Black = −1 (all 16 bits set); white = 0

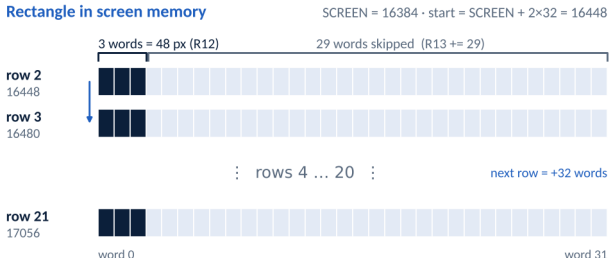


Fig. 14 Rectangle layout in screen memory (schematic).

Keyboard polling, drawing and clearing

1. **Poll.** (LOOP) executes @KBD, D=M; D;JNE jumps to SET_BLACK while a key is held, otherwise execution falls through to SET_WHITE.
2. **Colour.** SET_BLACK stores −1 in R12, SET_WHITE stores 0; both continue at FILL_SETUP.
3. **Set-up.** R13 = SCREEN + 64 and R14 = 20.
4. **Fill.** Per row, R15 = 3 and the WORD loop writes R12 to RAM[R13], increments R13 and decrements R15 until it is zero. Then R13 += 29 (next row) and R14 is decremented; after the last row control returns to (LOOP), so the box is redrawn or cleared continuously.

Register roles

R12	Colour: −1 (black) or 0 (white), set by the keyboard test
R13	Screen pointer: address of the next word to write
R14	Row counter: 20 down to 0
R15	Word counter: 3 down to 0 within each row

Listing 2 Box.asm from (ROW) to the end (comments omitted)

```
(ROW)
    @3
    D=A
    @R15
    M=D
(WORD)
    @R12
    D=M
    @R13
    A=M
    M=D
    @R13
    M=M+1
    @R15
    MD=M-1
    @WORD
    D;JGT
    @29
    D=A
    @R13
    M=D+M
    @R14
    MD=M-1
    @ROW
    D;JGT
    @LOOP
    0;JMP
```

CPU Emulator evidence

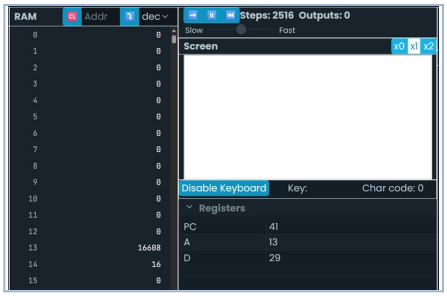


Fig. 15 No key pressed (step 2516): screen blank, R12 = 0.

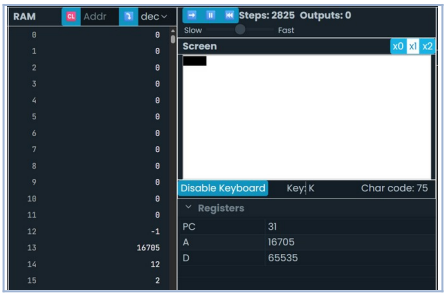


Fig. 16 Key "K" held (char code 75): R12 = −1; the black box shows at the top left, from row 2.

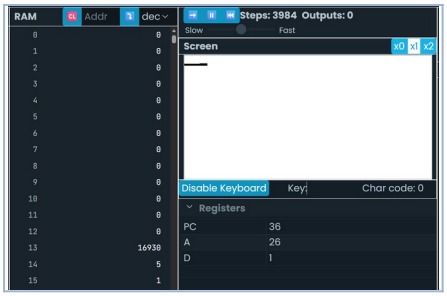


Fig. 17 Key released (char code 0): R12 = 0; the clear pass is under way (R14 = 5), so the last rows are still black.

Testing and results

- **Translation:** assembler.py (not the official assembler) turns Box.asm into Box.hack, 47 instructions; the first word, 0110000000000000, is @24576 (KBD). Re-running the assembler on Box.asm regenerates the supplied Box.hack.
- **Emulation:** run in the CPU Emulator, not the Hardware Simulator. Figs. 15–17 show the idle, drawing and clearing states.
- **Not performed:** a run of Box.hack on Computer.hdl; Part A validated the computer with the official programs only.

Mistake Encountered and Corrective Action

Initial emulator screenshots were taken mid-update, making the box look partially drawn or cleared. I let the program finish its 20-row pass and verified the stable states: the full box while a key was held, and a blank screen after release.

Conclusion. The capstone gives a complete path from HDL chips to a visible result. The computer passed all five official tests, the assembler matched the official assembler on four programs and passed five self-tests, and Box.hack, assembled by that assembler, draws and clears a 48 × 20-pixel box for K = 02 in the CPU Emulator. Running the same program directly on Computer.hdl would be the natural next check.